WhoIsIn

Интеллектуальный сервис сортировки фотографий по лицам

AI-powered Face Recognition + Clustering

InsightFace · HDBSCAN · FastAPI · React · PostgreSQL + pgvector

Проблема

📸

После мероприятия

Фотограф снял 500–1000 фото. Каждый участник хочет найти себя. Организатор вручную сортирует часами.

🎯

Решение: WhoIsIn

Загрузил фото → AI нашёл лица → сгруппировал по людям → участники находят себя по селфи за секунды

Боли

- Ручная сортировка = часы работы
- Участники ждут фото днями
- Нет удобного инструмента
- Платные решения = дорого

Ключевые фичи

- **Real-time pipeline** с WebSocket-прогрессом
- **Selfie search** — найди себя по фото
- **ZIP export** — скачай все свои фото
- **Privacy-first** — consent + удаление данных

01

AI-разработка

BMAD-METHOD · GitHub Issues · MCP Playwright

BMAD-METHOD: AI-driven Agile

Полный цикл от PRD до деплоя за 1 день. Claude Code выступает в роли целой команды разработки.

01	02	03	04	05	06	07	08
PRD	ADR (x7)	OpenAPI	DB Schema	UI/UX	Backend	Frontend	Deploy

Что такое BMAD-METHOD?

BMAD (BMad Agentic Development) — это AI-driven agile методология, где **Claude Code выступает в роли целой команды**. Вместо найма специалистов, один AI-агент переключается между ролями: PM пишет PRD, Architect принимает архитектурные решения (ADR), UX Designer создаёт дизайн-систему, Developer пишет код, QA тестирует. Всё построено на **spec-driven подходе**: сначала спецификация, потом реализация. 43 специализированных скилла с чек-листами обеспечивают качество на каждом этапе.

Роли AI-агента (Claude Code)

- **PM (John)** — написал PRD с требованиями и user stories
- **Architect (Winston)** — принял 7 ADR с обоснованием выбора
- **UX Designer (Sally)** — создал дизайн-систему и компоненты
- **Developer (Amelia)** — реализовал backend + frontend
- **QA (Quinn)** — протестировал через MCP Playwright

Spec-driven подход

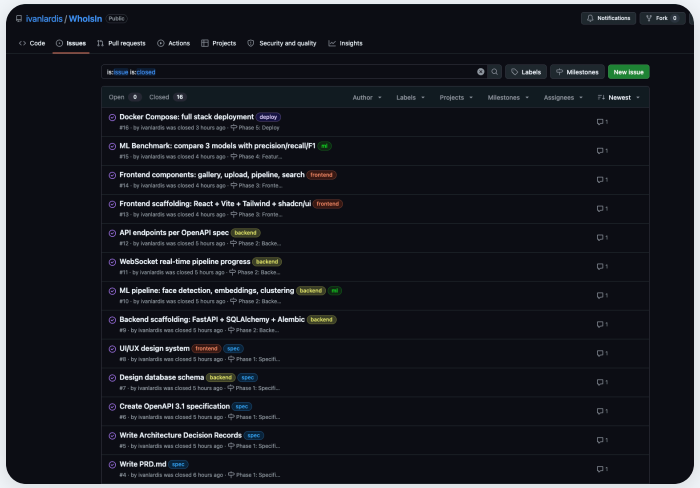
- **OpenAPI spec** — source of truth для API (сначала контракт, потом код)
- **orval** — автогенерация TypeScript клиента из OpenAPI
- **Pydantic** — валидация на бэкенде по схеме

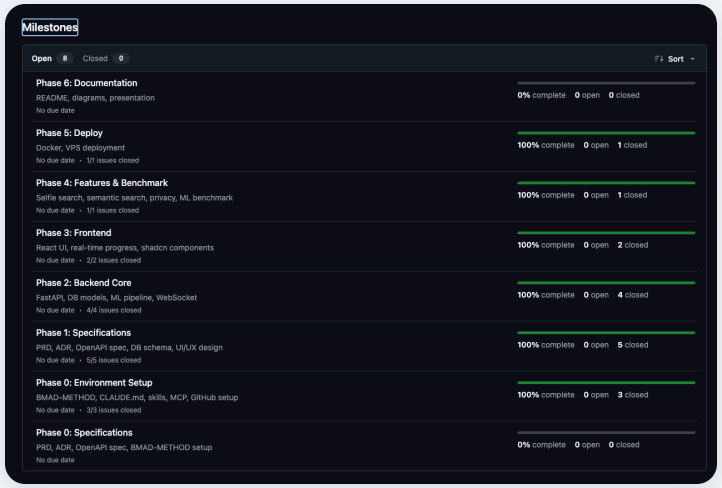
7 Architecture Decision Records

ADR-001	ML Model → InsightFace
ADR-002	Clustering → HDBSCAN
ADR-003	Database → PostgreSQL + pgvector
ADR-004	Frontend → React + shadcn/ui
ADR-005	Pipeline → FastAPI WebSocket
ADR-006	Descriptions → OpenRouter Gemini
ADR-007	Deploy → Docker + VPS

GitHub Issues: трекинг задач

16 issues, 7 milestones, 6 кастомных labels — всё закрыто





16/16 issues closed

Полное покрытие: от настройки окружения до деплоя на VPS

Labels

spec (5) backend (5) frontend (3) ml (2) deploy (1) dx (3)

Дизайн-система: UI/UX Pro Max

Кастомный Claude Code скилл для генерации design tokens

Стиль: Swiss Minimalism

Чистые линии, много пространства, фокус на контенте.
Выбран из 5 стилей через скилл UI/UX Pro Max.

Палитра

Indigo (primary) · Emerald (CTA) · Amber (warning) · Slate (text)

Типографика

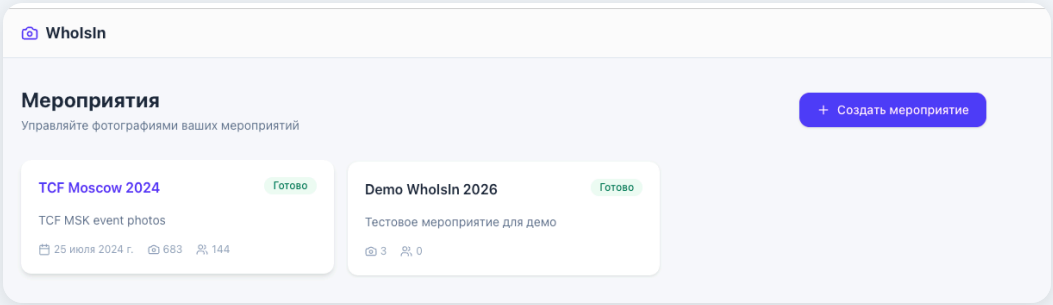
Space Grotesk — заголовки

DM Sans — основной текст

`JetBrains Mono — метрики`

Компоненты (7)

- **UploadZone** — drag & drop загрузка
- **PipelineProgress** — 5-шаговый прогресс
- **PersonCard** — аватар + статистика
- **PhotoGrid** — адаптивная сетка
- **PhotoLightbox** — полноэкранный просмотр
- **SelfieSearch** — поиск по лицу
- **ConsentBanner** — согласие GDPR



Тестирование: MCP Playwright

AI-агент сам тестирует приложение в реальном браузере — без тестовых скриптов

Что такое MCP?

Model Context Protocol — стандарт от Anthropic, дающий LLM доступ к внешним инструментам. Claude Code через MCP управляет **реальным браузером**: открывает страницы, кликает, заполняет формы, делает скриншоты.

Как это работает

- Подключение к браузеру через MCP Playwright
- Выполнение сценариев: навигация, клики, ввод
- Скриншоты + анализ DOM на каждом шаге
- AI сам решает, прошёл ли тест

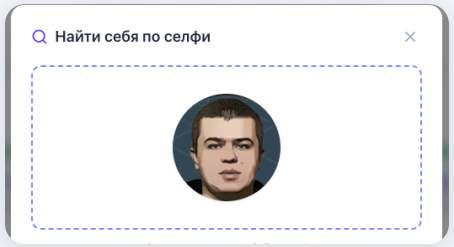
Покрытие

Создание ивента | Загрузка фото | Pipeline | Галерея | VPS деплой

```
// Claude Code + MCP Playwright
> browser_navigate("localhost:5173")
Page loaded: WhoIsIn
> browser_click(ref="create-event")
Modal opened
> browser_type(ref="name", "TCF")
> browser_click(ref="submit")
Event created: id=1
> browser_take_screenshot()
✔ Saved: event-created.png
```

Преимущества

Не нужно писать тесты — AI сам понимает что проверять. Тестировали и на localhost, и на VPS. 13 скриншотов проанализированы AI.



02

Выбор стека

Сравнение альтернатив и обоснование решений

ML: Выбор модели детекции лиц

Бенчмарк 3 моделей на реальных данных

МОДЕЛЬ	СКОРОСТЬ (CPU)	РАЗМЕР	КАЧЕСТВО	EMBEDDINGS	ВЕРДИКТ
InsightFace buffalo_sc	**~150ms/фото**	**16 MB**	**Высокое**	**ArcFace 512-dim**	**✔ Выбрано**
InsightFace buffalo_l	~450ms/фото	127 MB	Очень высокое	ArcFace 512-dim	Слишком медленно для CPU
dlib HOG + face_recognition	~200ms/фото	~100 MB	Среднее	128-dim	Устаревшие embeddings
MTCNN + FaceNet	~300ms/фото	~90 MB	Хорошее	128-dim	Сложная интеграция

Почему buffalo_sc: 3x быстрее buffalo_l при сравнимом качестве на CPU-сервере. ArcFace 512-dim embeddings дают лучшую кластеризацию, чем 128-dim альтернативы.

Кластеризация: HDBSCAN vs альтернативы

Группировка лиц без заранее известного числа людей

АЛГОРИТМ	НУЖЕН K?	ШУМ	КЛАСТЕРЫ РАЗНОГО РАЗМЕРА	СЛОЖНОСТЬ
HDBSCAN	**Нет**	**Да**	**Да**	**O(n log n)**
K-Means	Да (K)	Нет	Нет	O(nKi)
DBSCAN	Нет, но нужен eps	**Да**	Нет	O(n log n)
Agglomerative	Да (threshold)	Нет	**Да**	$O(n^3)$

Почему HDBSCAN: Не требует заранее знать количество людей. Автоматически выделяет выбросы (нечёткие лица). min_cluster_size=2 позволяет группировать даже людей с 2 фото.

Полный стек технологий

Backend

Python 3.12 | FastAPI | SQLAlchemy async | Pydantic v2

15 REST endpoints + WebSocket для real-time прогресса pipeline

Frontend

React 19 | TypeScript | Vite | Tailwind v4 | shadcn/ui

Typed API client (orval) + React Query + WebSocket hooks

Database

PostgreSQL 16 | pgvector

Единая БД: реляционные данные + векторный поиск по embeddings (cosine similarity)

Deploy

Docker Compose | Nginx | VPS Ubuntu

3 контейнера: db, backend, frontend. Reverse proxy + WebSocket + static caching

ML Pipeline

InsightFace | HDBSCAN | OpenRouter

Детекция → Vectorize → Кластеризация → Описания (Gemini Flash)

Почему не...

Django → FastAPI быстрее для async ML pipeline
MongoDB → pgvector = vector + relational в одной БД
Next.js → SPA + orval codegen проще для API-first
Kubernetes → overkill для одного VPS

03

Продукт

Архитектура, pipeline, результаты

Архитектура системы

```
Frontend (React 19)              Backend (FastAPI)              Database

┌──────────────────┐             ┌──────────────────┐           ┌──────────────────┐
│ React + Vite     │   REST      │ 15 API endpoints │   SQL     │ PostgreSQL 16    │
│ Tailwind + shadcn │ ──────────► │ WebSocket manager │ ────────► │ + pgvector       │
│ React Query      │ ◄──────────  │ Pipeline orchestrator│ ◄──────── │ 7 tables         │
│ WebSocket hook   │   WS        │ Face detector    │           │ 512-dim vectors  │
└──────────────────┘             │ Face clusterer   │           └──────────────────┘
                                 └──────────────────┘
                                          │
                                          ▼
                                 ┌──────────────────┐
                                 │ ML Pipeline      │
                                 │ InsightFace      │
                                 │ HDBSCAN          │
                                 │ OpenRouter API   │
                                 └──────────────────┘
```

ML Pipeline (5 фаз)

🔍 **Детекция лиц** → 🧬 **Vectorize** ArcFace 512-dim → 👥 **Кластеризация** HDBSCAN → 📝 **Описания** Gemini Flash → ✔ **Готово**

Результаты на реальных данных

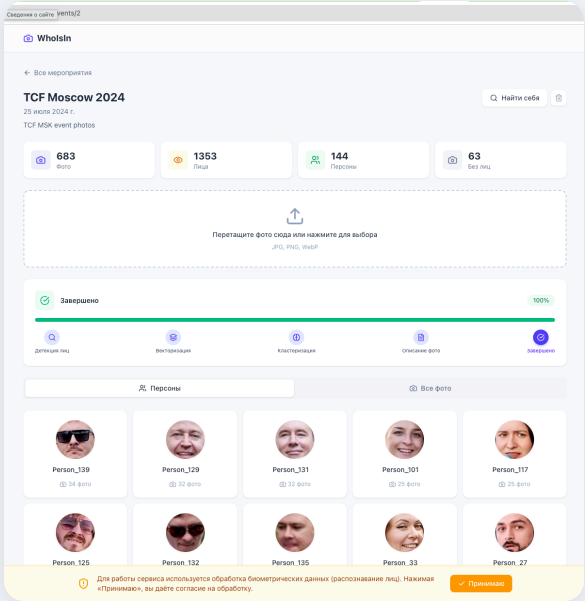
TCF Moscow 2024 — 682 фотографии с мероприятия

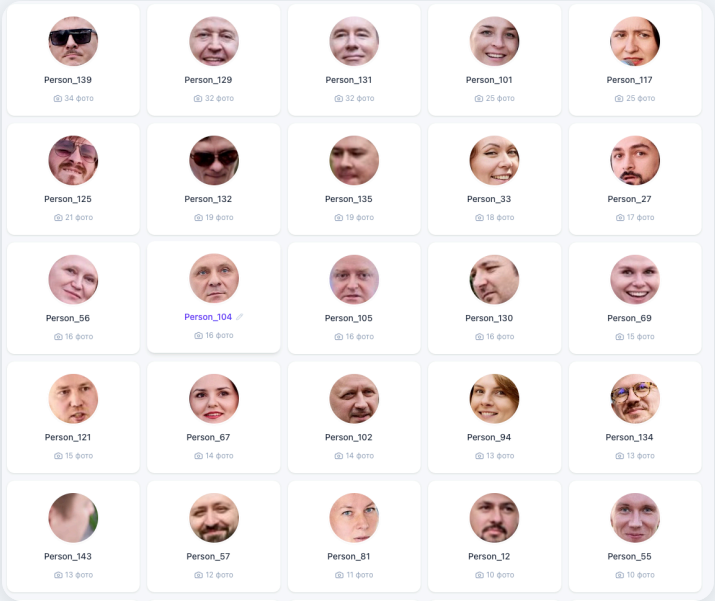
683
Фото загружено

1353
Лиц найдено

144
Персон определено

63
Фото без лиц





Demo: http://185.46.11.151/events/2

Статистика проекта

4228	19	7	13
Строк кода	Коммитов	ADR	Скриншотов

Backend (Python)

- 2160 строк, 20 файлов
- 15 REST endpoints
- WebSocket для real-time
- Async SQLAlchemy + pgvector

Frontend (TypeScript)

- 2068 строк, 18 файлов
- 7 UI-компонентов
- 3 страницы (SPA)
- Typed API + React Query

Инфра

- 3 Docker-сервиса
- Nginx reverse proxy
- VPS Ubuntu 22.04
- GitHub CI-ready

AI-accelerated: Полный цикл разработки (PRD → ADR → Schema → Code → Deploy) выполнен за ~1 день с помощью Claude Code + BMAD-METHOD + MCP.

WhoIsIn

Загрузил фото → AI нашёл лица → Каждый нашёл себя

Demo: 185.46.11.151

GitHub: ivanlardis/WhoIsIn

InsightFace | HDBSCAN | pgvector | FastAPI | React 19 | WebSocket | Docker | MCP Playwright | BMAD-METHOD